# Development Guide

Setup, conventions, and contribution workflow

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

This guide describes how to set up a working environment for the NLP search engine, the conventions the codebase follows, and the steps required to add new features such as semantic search, analytics tracking, autocomplete, faceted search, or incremental indexing. The supplied Makefile and Docker image cover the common day-to-day operations.

ABSTRACT

This guide describes how to set up a working environment for the NLP search engine, the conventions the codebase follows, and the steps required to add a new ranking model or front-end. The supplied Makefile and Docker image cover the common day-to-day operations.

## 1. Prerequisites

- Python 3.11 or newer.
- `git`.
- `make` (optional, but the supplied Makefile shortcuts are convenient).

## 2. Getting Started

```
git clone https://github.com/Hamza35779/NLP-AI-Search-Engine.git
cd NLP-AI-Search-Engine

python -m venv .venv
# Linux/macOS: source .venv/bin/activate
# Windows (PS): .venv\Scripts\Activate.ps1

pip install -e ".[dev]"
```

The editable install lets you edit modules under `src/` and have the changes picked up without reinstalling the package.

## 3. Common Tasks

| Task | Command |
|---|---|
| Run the test suite | pytest -q (or: make test) |
| Run a one-off CLI search | python cli.py "your query" |
| Start the web UI | python app.py (or: make run) |
| Rebuild the index | python scripts/build_index.py |
| Rebuild PDF docs | python scripts/build_docs.py |
| Lint with Ruff | make lint |

| Task | Command |
|---|---|
| Format with Black | make format |
| Build the Docker image | make docker |
| View Prometheus metrics | curl http://127.0.0.1:5000/metrics |
| Batch search | curl -X POST http://127.0.0.1:5000/api/batch-search |

## 4. Repository Layout

```
nlp-search-engine/
app.py Flask UI (with new endpoints)
cli.py Argparse CLI
requirements.txt Python dependencies
pyproject.toml Packaging + tool config (v0.3.0)
Dockerfile Production container
Makefile Common commands
README.md This file
LICENSE MIT License
src/ Engine source (every file <200 lines)
■■■ search_engine.py Main facade (with analytics, metrics)
■■■ semantic_search.py Semantic search (TF-IDF vectors)
■■■ analytics.py Search analytics tracking
■■■ autocomplete.py Query autocomplete suggestions
■■■ persistence.py SQLite backend (optional)
■■■ config.py Configuration (v0.3.0)
templates/ Jinja2 templates (with history, bookmarks)
static/ CSS, JS (app.js), and images
■■■ js/ JavaScript (history, bookmarks, theme)
tests/ Pytest suite (25 tests)
scripts/ Build/indexing scripts
examples/ Usage examples
data/ Sample corpus
```

## 5. Coding Conventions

- Each source file stays under 200 lines (split when necessary).
- Type hints everywhere; modules use `from __future__ import annotations`.
- Public APIs are exposed through the package facade in `src/__init__.py`.
- Comments only when the why is non-obvious; naming should make the what self-explanatory.

## 6. Adding a New Ranking Model

The Ranker is intentionally thin so a new scoring function is a small change.

- Implement a class in `src/<your_model>.py` exposing `score(query_terms) -> List[Tuple[int, float]]`.
- Register it in `src/ranker.py`: extend `VALID_MODELS`, add a lazy property, and dispatch in `Ranker.rank`.
- Update `docs/ranking-models.pdf` and add a unit test under `tests/`.

## 7. Adding a New Front-end

The `SearchEngine` class is framework-agnostic. The minimal sequence is:

```
from src.config import EngineConfig
from src.search_engine import SearchEngine

engine = SearchEngine(EngineConfig(), model="bm25")
engine.index_corpus("data/sample_docs")
results = engine.search("your query", top_k=5)
```

## 8. Testing Strategy

- Unit tests sit next to the module they exercise in `tests/test_*.py`.
- End-to-end tests drive the SearchEngine through realistic queries (see `tests/test_search.py`).
- The suite must remain fast (well under a second locally) so the contributor feedback loop stays painless.

## 9. Releasing

- Bump `__version__` in `src/__init__.py` and the version in `pyproject.toml`.
- Move the Unreleased section in the changelog under the new version.
- Tag the commit (`git tag -a vX.Y.Z -m "Release vX.Y.Z"`) and push the tag.
- CI runs the test suite on the tagged commit.